

Instituto Federal do Rio Grande do Norte
Campus Canguaretama
Tecnologia em Sistemas para Internet
Sistemas Operacionais Abertos

**PROJETO FINAL**

# Ferramenta de diagnóstico e auditoria de servidores Linux

*Desenvolvimento de uma aplicação em Shell Script*

## 1. Apresentação e contextualização

Imagine que uma empresa possui vários servidores Linux responsáveis por hospedar aplicações, armazenar arquivos e fornecer serviços de rede. A equipe de infraestrutura precisa verificar periodicamente se esses servidores estão funcionando corretamente.

Entre as atividades comuns estão:

- Verificar se as interfaces de rede estão configuradas e ativas.
- Conferir os endereços IP e as rotas configuradas.
- Testar a conectividade com outros equipamentos.
- Verificar se determinados serviços estão em execução.
- Analisar registros de acesso e identificar erros.
- Gerar relatórios de diagnóstico.
- Registrar as verificações realizadas para consultas posteriores.

Atualmente, essas tarefas podem ser executadas manualmente, utilizando vários comandos diferentes do Linux. O objetivo deste projeto é desenvolver um programa em Shell Script que automatize parte dessas atividades, reunindo as verificações em uma única ferramenta de linha de comando.

## 2. Objetivo do projeto

Desenvolver uma ferramenta chamada `netadmin.sh`, executada pelo terminal, que receba opções e argumentos, realize verificações de diagnóstico e auditoria de um servidor Linux e apresente os resultados de maneira organizada. O programa deverá ser modular, validar entradas, tratar situações de erro e permitir a geração de relatórios.

## 3. Funcionamento esperado e interface de utilização

A ferramenta deverá aceitar opções que determinem quais operações serão realizadas.

| Opção | Finalidade |
|---|---|
| `-i` | Exibir informações do servidor e das interfaces de rede. |
| `-p DESTINO` | Testar a conectividade com um destino. |
| `-P ARQUIVO` | Testar a conectividade com todos os destinos listados em um arquivo. |
| `-s` | Listar serviços e portas em estado de escuta. |
| `-c PORTAS` | Verificar uma ou mais portas locais específicas (separadas por vírgula). |
| `-l ARQUIVO` | Analisar um arquivo de log. |
| `-o ARQUIVO` | Salvar os resultados em um relatório. |
| `-v` | Ativar o modo detalhado. |
| `-h` | Exibir a ajuda de utilização. |

O programa deverá informar quando uma opção exigir um argumento e esse argumento não tiver sido fornecido.

## 4. Funcionalidades obrigatórias

### Módulo 1 — Informações do servidor e interfaces de rede

O programa deverá apresentar informações básicas sobre o servidor:

- Nome da máquina e data/horário da execução.
- Interfaces de rede disponíveis e estado das interfaces.
- Endereços IP configurados.
- Rotas de rede configuradas.

Comandos que podem ser utilizados:

```
hostname
date
ip address
ip link
ip route
```

Não basta exibir a saída de todos os comandos listados acima. O programa deverá organizar as informações e, quando possível, destacar situações relevantes, como uma interface desativada ou sem endereço IP configurado. Por exemplo, suponha que os comandos `ip link`, `ip address` e `ip route` retornem informações como as seguintes:

```
ip link:
enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP>
enp0s8: <BROADCAST,MULTICAST>
enp0s9: <BROADCAST,MULTICAST,UP,LOWER_UP>

ip address:
enp0s3: 192.168.1.20/24
enp0s9: (nenhum endereço IP configurado)

ip route:
192.168.1.0/24 dev enp0s3
```

Ao utilizar a opção `-i`, O NetAdmin deverá apresentar uma saída semelhante a seguinte:

```
=== DIAGNOSTICO DE REDE ===

INTERFACES DE REDE

enp0s3
  Estado: ATIVA
  Endereço IPv4: 192.168.1.20/24

enp0s8
  Estado: DESATIVADA
  Endereço IPv4: NÃO CONFIGURADO
  ATENÇÃO: Interface desativada.

enp0s9
  Estado: ATIVA
  Endereço IPv4: NÃO CONFIGURADO
  ATENÇÃO: Interface ativa, mas sem endereço IP configurado.
```

```
ROTAS DE REDE

Rota padrão: NÃO CONFIGURADA
  ATENÇÃO: O servidor pode não conseguir acessar redes externas.

=== RESUMO DO DIAGNOSTICO ===

Interfaces encontradas: 3
Interfaces ativas: 2
Interfaces desativadas: 1
Interfaces sem endereço IPv4: 2
Rota padrão: NÃO CONFIGURADA

Foram identificadas 3 situações que merecem atenção.
```

Ao passar a opção `-v` (modo detalhado), a saída deve conter mais informações, como no exemplo a seguir:

```
=== INFORMACOES DO SERVIDOR (MODO DETALHADO) ===

INFORMACOES DO SISTEMA
Hostname: srv-web
Sistema operacional: Linux
Kernel: 6.8.0-45-generic
Tempo ligado: 3 dias, 4 horas e 20 minutos

INTERFACES DE REDE

enp0s3
  Estado: ATIVA
  Endereco MAC: 08:00:27:12:34:56
  IPv4: 192.168.1.20/24
  IPv6: fe80::a00:27ff:fe12:3456/64
  MTU: 1500

enp0s8
  Estado: DESATIVADA
  Endereco MAC: 08:00:27:65:43:21
  IPv4: NAO CONFIGURADO
```

```
  IPv6: NAO CONFIGURADO
  MTU: 1500
  ATENCAO: Interface desativada.

enp0s9
  Estado: ATIVA
  Endereco MAC: 08:00:27:ab:cd:ef
  IPv4: NAO CONFIGURADO
  IPv6: fe80::a00:27ff:feab:cdef/64
  MTU: 1500
  ATENCAO: Interface sem endereco IPv4.

ROTAS DE REDE
  192.168.1.0/24 dev enp0s3 proto kernel scope link
  127.0.0.0/8 dev lo scope host

Rota padrao: NAO CONFIGURADA

COMANDOS UTILIZADOS
  hostname
  date
  uname -r
  uptime
  ip link
  ip address
  ip route

DIAGNOSTICO
  1. Interface enp0s8 desativada.
  2. Interface enp0s8 sem endereco IPv4.
  3. Interface enp0s9 sem endereco IPv4.
  4. Rota padrao nao configurada.
```

A versão detalhada apresenta também endereço MAC, IPv6, MTU, informações do sistema, tabela de rotas completa e os comandos utilizados.

## Módulo 2 — Testes de conectividade

O programa deverá permitir testar a comunicação com um ou mais destinos informados pelo usuário.

```
./netadmin.sh -p 192.168.1.1
./netadmin.sh -p google.com
```

O programa deverá:

- Verificar se o usuário informou um destino.
- Executar um teste de conectividade.
- Identificar se houve resposta.
- Apresentar o resultado de maneira compreensível.
- Registrar o resultado no relatório, quando solicitado.

Como extensão natural, o programa poderá aceitar um arquivo contendo vários destinos, um por linha:

```
192.168.1.1
192.168.1.10
google.com
```

Nesse caso, deverá percorrer os destinos e testar cada um deles, tratando individualmente os resultados e eventuais falhas.

Exemplo de execução normal:

```
=== TESTE DE CONECTIVIDADE ===

Destino: google.com
Resultado: SUCESSO

Pacotes enviados: 4
Pacotes recebidos: 4
Perda: 0%
Tempo medio: 12.5 ms
```

Exemplo de execução em modo detalhado (`-v`):

```
=== TESTE DE CONECTIVIDADE (MODO DETALHADO) ===

Destino informado: google.com
Endereco IP resolvido: 142.250.219.14
```

```
EXECUCAO DO TESTE
  Tentativa 1: RESPOSTA - 12.1 ms
  Tentativa 2: RESPOSTA - 13.0 ms
  Tentativa 3: RESPOSTA - 11.8 ms
  Tentativa 4: RESPOSTA - 13.1 ms

ESTATISTICAS
  Pacotes enviados: 4
  Pacotes recebidos: 4
  Pacotes perdidos: 0
  Perda: 0%
  Tempo minimo: 11.8 ms
  Tempo medio: 12.5 ms
  Tempo maximo: 13.1 ms

RESULTADO: SUCESSO
```

A versão detalhada apresenta as estatísticas de cada destino individualmente, incluindo o endereço IP resolvido quando o destino é um nome de domínio.

## Módulo 3 — Verificação de serviços e portas

O programa deverá identificar serviços e portas em estado de escuta no próprio servidor. Um comando que pode auxiliar nessa tarefa é:

```
ss -tuln
```

A funcionalidade poderá:

- Listar as portas em estado de escuta (opção `-s`).
- Verificar se uma porta específica está aberta (opção `-c PORTA`).
- Permitir verificar uma lista de portas, separadas por vírgula (`opção -c PORTA1,PORTA2,PORTA3...`).
- Informar quando determinada porta não estiver disponível.

```
./netadmin.sh -s
./netadmin.sh -c 80
./netadmin.sh -c 22,80,443
```

Exemplo de execução com `-s` em modo normal:

```
=== SERVICOS E PORTAS LOCAIS ===

PROTOCOLO   ENDERECO LOCAL       PORTA
tcp         0.0.0.0              22
tcp         0.0.0.0              80
tcp         127.0.0.1            3306
udp         0.0.0.0              53

Total de sockets encontrados: 4
```

Exemplo de execução em modo detalhado (-v):

```
=== SERVICOS E PORTAS LOCAIS (MODO DETALHADO) ===

SOCKETS TCP
PROTOCOLO   ENDERECO LOCAL       PORTA
tcp         0.0.0.0              22
tcp         0.0.0.0              80
tcp         127.0.0.1            3306

SOCKETS UDP
PROTOCOLO   ENDERECO LOCAL       PORTA
udp         0.0.0.0              53

DETALHAMENTO
  0.0.0.0:22 - Escuta em todas as interfaces IPv4.
  0.0.0.0:80 - Escuta em todas as interfaces IPv4.
  127.0.0.1:3306 - Disponivel somente localmente.
  0.0.0.0:53 - Escuta em todas as interfaces IPv4.

RESUMO
  Sockets TCP: 3
  Sockets UDP: 1
  Total: 4

COMANDO UTILIZADO
  ss -tuln
```

A versão detalhada diferencia portas disponíveis em todas as interfaces daquelas disponíveis somente localmente e separa os sockets TCP e UDP.

Exemplo de saída para a execução de `./netadmin.sh -c 22,80,443`:

```
=== VERIFICACAO DE PORTAS ===

Porta 22: ABERTA (TCP)
Porta 80: ABERTA (TCP)
Porta 443: NAO ENCONTRADA

Resumo: 2 encontradas; 1 nao encontrada.
```

## Módulo 4 — Análise de arquivos de log

O programa deverá receber um arquivo de log e produzir um resumo de sua atividade. Por exemplo:

```
./netadmin.sh -l access.log
```

A análise poderá identificar:

- Quantidade total de registros.
- Quantidade de respostas HTTP 200.
- Quantidade de erros 404.
- Quantidade de erros 500.
- Endereços IP que mais acessaram o servidor.
- Endereços IP que geraram mais erros.

Ferramentas que podem ser utilizadas:

```
grep
cut
sort
uniq
wc
```

Exemplo ilustrativo de resultado (em modo normal):

```
================================
       RELATÓRIO DE ACESSOS
================================
```

```
Total de requisições: 1500

Respostas HTTP:
  200: 1320
  404: 120
  500: 60

Endereços com mais acessos:
  192.168.1.10: 450
  192.168.1.15: 320
  192.168.1.20: 210
```

Exemplo em modo detalhado (`-v`):

```
=== ANALISE DE LOG (MODO DETALHADO) ===

Arquivo: logs/acessos_basico.log
Registros processados: 10
Linhas invalidas: 0

RESPOSTAS HTTP POR CODIGO
200: 5
302: 1
403: 1
404: 2
500: 1

CLASSIFICACAO
2xx: 5 (50%)
3xx: 1 (10%)
4xx: 3 (30%)
5xx: 1 (10%)

REQUISICOES POR IP
192.0.2.10: 4
192.0.2.11: 2
192.0.2.12: 2
192.0.2.13: 1
192.0.2.14: 1
```

```
REQUISICOES POR RECURSO
/: 2
/login: 2
/produtos: 2
/admin: 1
/nao-existe: 1
/relatorios: 1
/api/pedidos: 1

REQUISICOES POR METODO
GET: 8
POST: 2

ERROS HTTP
Respostas 4xx: 3
Respostas 5xx: 1
Total: 4

HORARIO
Primeira requisicao: 09:00:00
Ultima requisicao: 09:09:00
```

A versão detalhada apresenta as contagens individuais por código HTTP, todos os IPs e recursos encontrados, distribuição dos métodos e estatísticas adicionais.

## Módulo 5 — Relatório e histórico de execução

O programa deverá permitir salvar os resultados em um arquivo. Por exemplo:

```
./netadmin.sh -o auditoria.txt
```

O relatório deverá conter:

- Data e horário da execução.
- Nome do servidor.
- Verificações realizadas.
- Resultados obtidos.

- Problemas identificados.

O programa deverá considerar situações como:

- O diretório de saída não existe.
- O arquivo não pode ser escrito.
- O arquivo de entrada não existe.
- Uma verificação não pode ser executada.
- Um comando necessário não está instalado.

## 5. Requisitos técnicos e conteúdos que deverão ser demonstrados4

O projeto não será avaliado apenas pelo resultado final. O código deverá demonstrar que os alunos compreenderam e conseguem aplicar os recursos da linguagem estudados durante o semestre.

| Conteúdo estudado | Como deverá aparecer no projeto |
|---|---|
| **Argumentos posicionais** | Receber arquivos, endereços IP, portas ou outros parâmetros. |
| **getopts** | Interpretar opções de execução, como -i, -p, -l, -o e -v. |
| **OPTIND e shift** | Separar as opções dos argumentos posicionais restantes. |
| **case** | Interpretar opções e selecionar operações. |
| **if** | Tomar decisões com base nos resultados das verificações. |
| **test** | Validar arquivos, diretórios, permissões e argumentos. |
| **Laços de repetição** | Percorrer listas de destinos, portas ou registros. |
| **Variáveis** | Armazenar parâmetros, resultados e configurações. |
| **Funções** | Organizar o programa em módulos reutilizáveis. |
| **Parâmetros de funções** | Passar informações entre as diferentes funcionalidades. |
| **echo e printf** | Apresentar informações e produzir relatórios. |
| **Redirecionamentos** | Registrar saídas e erros em arquivos. |
| **Pipes** | Encadear comandos para processar informações. |
| **grep, cut, sort, uniq, wc** | Processar arquivos de log. |
| **Substituição de comandos** | Capturar resultados de comandos e funções. |
| **trap** | Tratar interrupções e remover arquivos temporários. |
| **Depuração** | Testar o programa utilizando **sh -n** e **sh -x**. |

## 6. Critérios de avaliação

A avaliação considerará tanto o funcionamento quanto a qualidade da implementação, de acordo com a seguinte tabela de critérios:

| Critério | Pontuação |
|---|---|
| Planejamento e interface de opções e argumentos | 10 |
| Implementação correta das funcionalidades | 25 |
| Utilização adequada das estruturas do Shell Script | 25 |
| Modularização com funções e organização do código | 10 |
| Tratamento de erros e situações excepcionais | 10 |
| Relatórios, redirecionamentos e processamento de arquivos | 10 |
| Documentação | 10 |
| **TOTAL** | **100** |

**Atenção: não basta executar um comando Linux dentro de um script para obter todos os pontos de uma funcionalidade. Por exemplo, no módulo de conectividade, não bastaria escrever ping "$1". O programa deverá validar os argumentos, interpretar o resultado, tratar falhas e apresentar uma saída compreensível.**